

Data Mining – Homework#2: Association Rule Mining and Time Series Analysis

Razieh Doregard – 40435056

Loading and Initial Cleaning

```
<bound method NDFrame.head of
0      536365      85123A  WHITE HANGING HEART T-LIGHT HOLDER      6
1      536365      71053      WHITE METAL LANTERN      6
2      536365      84406B      CREAM CUPID HEARTS COAT HANGER      8
3      536365      84029G  KNITTED UNION FLAG HOT WATER BOTTLE      6
4      536365      84029E      RED WOOLLY HOTTIE WHITE HEART.      6
...      ...      ...      ...      ...
541904      581587      22613      PACK OF 20 SPACEBOY NAPKINS      12
541905      581587      22899      CHILDREN'S APRON DOLLY GIRL      6
541906      581587      23254      CHILDRENS CUTLERY DOLLY GIRL      4
541907      581587      23255      CHILDRENS CUTLERY CIRCUS PARADE      4
541908      581587      22138      BAKING SET 9 PIECE RETROSPOT      3

      InvoiceDate  UnitPrice  CustomerID      Country
0      2010-12-01 08:26:00      2.55      17850.0  United Kingdom
1      2010-12-01 08:26:00      3.39      17850.0  United Kingdom
2      2010-12-01 08:26:00      2.75      17850.0  United Kingdom
3      2010-12-01 08:26:00      3.39      17850.0  United Kingdom
4      2010-12-01 08:26:00      3.39      17850.0  United Kingdom
...      ...      ...      ...      ...
541904      2011-12-09 12:50:00      0.85      12680.0      France
541905      2011-12-09 12:50:00      2.10      12680.0      France
541906      2011-12-09 12:50:00      4.15      12680.0      France
541907      2011-12-09 12:50:00      4.15      12680.0      France
541908      2011-12-09 12:50:00      4.95      12680.0      France
...
7      Country      541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
None
```

Handle Missing Values

```
Before removing missing values:
InvoiceNo      0
StockCode      0
Description    1454
Quantity      0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64
```

I removed all items without descriptions:

```
After removing missing values:  
InvoiceNo      0  
StockCode      0  
Description    0  
Quantity       0  
InvoiceDate    0  
UnitPrice      0  
CustomerID    133626  
Country        0  
dtype: int64
```

Remove invalid records

1- Removed the items with invalid quantities

```
bad_cnt = df[(df["Quantity"] <= 0)].shape[0]  
print("Rows with zero or negative quantity BEFORE:", bad_cnt)  
  
df_clean = df[df["Quantity"] > 0]  
after_cnt = df_clean[(df_clean["Quantity"] <= 0)].shape[0]  
print("Rows with zero or negative quantity AFTER:", after_cnt)
```

```
Rows with zero or negative quantity BEFORE: 9762  
Rows with zero or negative quantity AFTER: 0
```

2- Removed items with invalid prices

```
bad_cnt = df[(df["UnitPrice"] <= 0)].shape[0]  
print("Rows with zero or negative UnitPrice BEFORE:", bad_cnt)  
  
df_clean = df[df["UnitPrice"] > 0]  
after_cnt = df_clean[(df_clean["UnitPrice"] <= 0)].shape[0]  
print("Rows with zero or negative quantity AFTER:", after_cnt)
```

```
Rows with zero or negative UnitPrice BEFORE: 1063  
Rows with zero or negative quantity AFTER: 0
```

3- Removed cancelled purchases:

```
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
```

4- Removing invalid stock codes and normalizing descriptions:

```
Non-product stock codes found:
- POST: 1252 records
- DOT: 709 records
- M: 572 records
- C2: 143 records
- D: 77 records
- S: 63 records
- BANK CHARGES: 37 records
- AMAZONFEE: 34 records
- CRUK: 16 records
- PADS: 4 records
- B: 3 records
Removed 2910 non-product stock code records

--- 3. Description Standardization ---
All descriptions converted to lowercase and whitespace stripped
Anomalous description patterns found:
- 'adjust': 31 records
- 'damages': 61 records
- 'lost': 6 records
- 'incorrect': 7 records
- 'thrown away': 20 records
- 'crushed': 5 records
- 'wet': 17 records
- 'rusty': 8 records
...
UnitPrice      0
CustomerID    131963
Country        0
dtype: int64
```

Data Type Conversion

Converted the type of invoice date to python dateTime data type:

```
print("BEFORE: ", df["InvoiceDate"].dtype)
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
print("AFTER: ", df["InvoiceDate"].dtype)
```

```
BEFORE:  datetime64[ns]
AFTER:  datetime64[ns]
```

Feature Engineering

Created a new column in the data frame called “TotalAmount” which is the product of the 2 columns of UnitPrice and Quantity to find the total amount gained from a purchased product.

```
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
print(df["Quantity"].head, df["UnitPrice"].head, df["TotalAmount"].head)
```

<bound method NDFrame.head of 0 6

1	6
2	8
3	6
4	6
..	
541904	12
541905	6
541906	4
541907	4
541908	3

Name: Quantity, Length: 540455, dtype: int64 <bound method NDFrame.head of 0

1	3.39
2	2.75
3	3.39
4	3.39
...	
541904	0.85
541905	2.10
541906	4.15

Grouping data points by the attribute “InvoiceNo” to keep track of all product items that were purchased together; the information gained from this grouping will be further used to find the frequent item sets.

```
original_count = df.shape[0]
df_grouped = df.groupby(["InvoiceNo", "Description"], as_index=False)["Quantity"].sum()

new_count = df_grouped.shape[0]
print("Before:", original_count)
print("After:", new_count)

print(df_grouped.columns)
```

Before: 531167
After: 520448
Index(['InvoiceNo', 'Description', 'Quantity'], dtype='object')

Removed the products with sell counts < 10 to keep track of valuable and frequent products only.

```
product_counts = df_grouped["Description"].value_counts()
frequent_products = product_counts[product_counts >= 10].index
df_filtered = df_grouped[df_grouped["Description"].isin(frequent_products)]
print(df_filtered["Description"].value_counts().min())
```

10

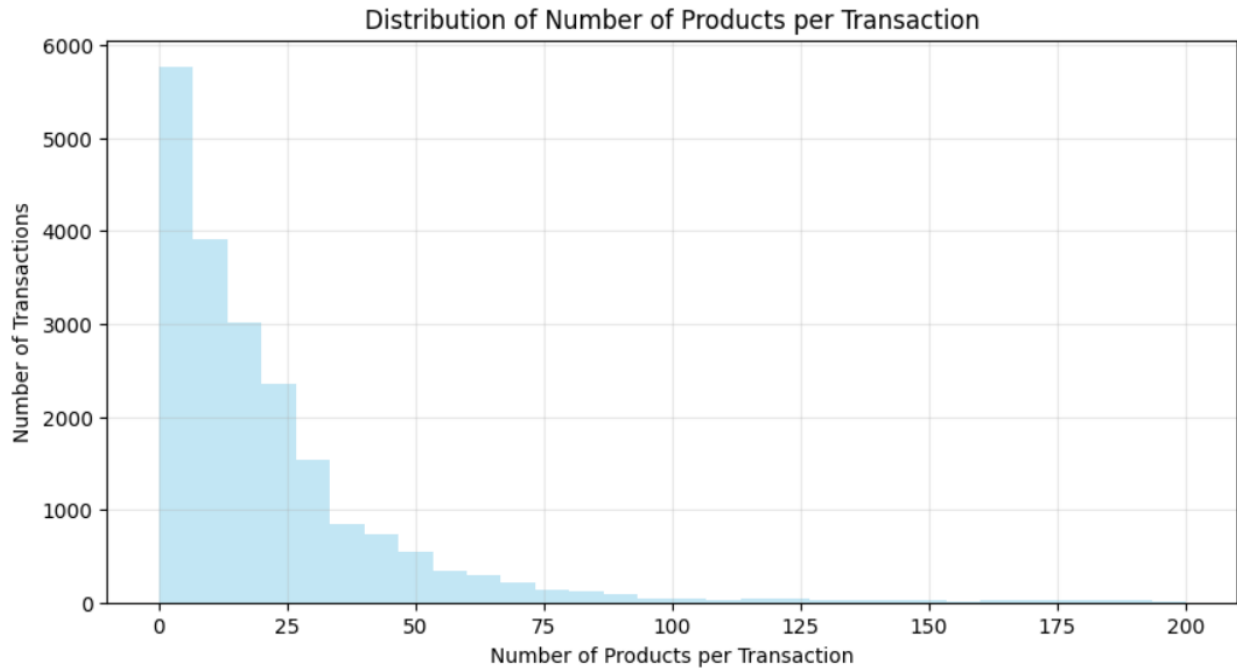
Preprocessing for Association Rule Mining

Creating baskets from the grouped attributes to start with the association rule mining process:

```
baskets = df_grouped.groupby("InvoiceNo")["Description"].apply(list).reset_index()
print(baskets.head)
```

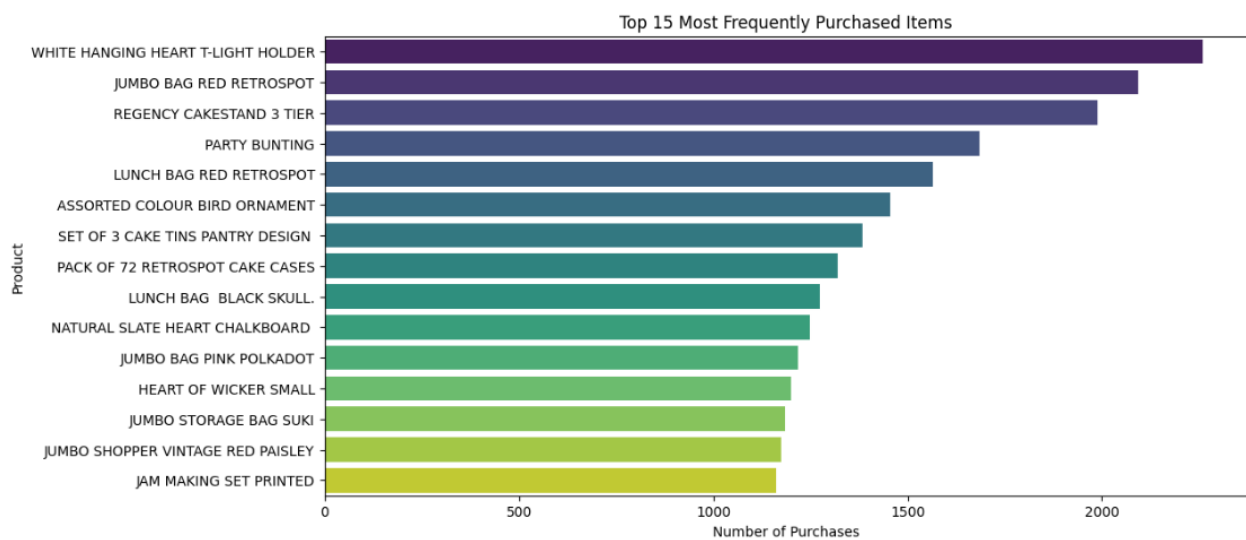
		InvoiceNo	Description
<bound method NDFrame.head of			
0	536365	[white hanging heart t-light holder, white met...	
1	536366	[hand warmer union jack, hand warmer red polka...	
2	536367	[assorted colour bird ornament, poppy's playho...	
3	536368	[jam making set with jars, red coat rack paris...	
4	536369	[bath building block word]	
...	...	...	
19731	581583	[lunch bag red retrospot, 6 chocolate love hea...	
19732	581584	[red flock love heart photo frame, 6 chocolate...	
19733	581585	[black tea towel classic design, assorted bott...	
19734	581586	[large cake stand hanging strawberry, set of 3...	
19735	581587	[circus parade lunch box, plasters in tin circ...	

EDA for Association Rules

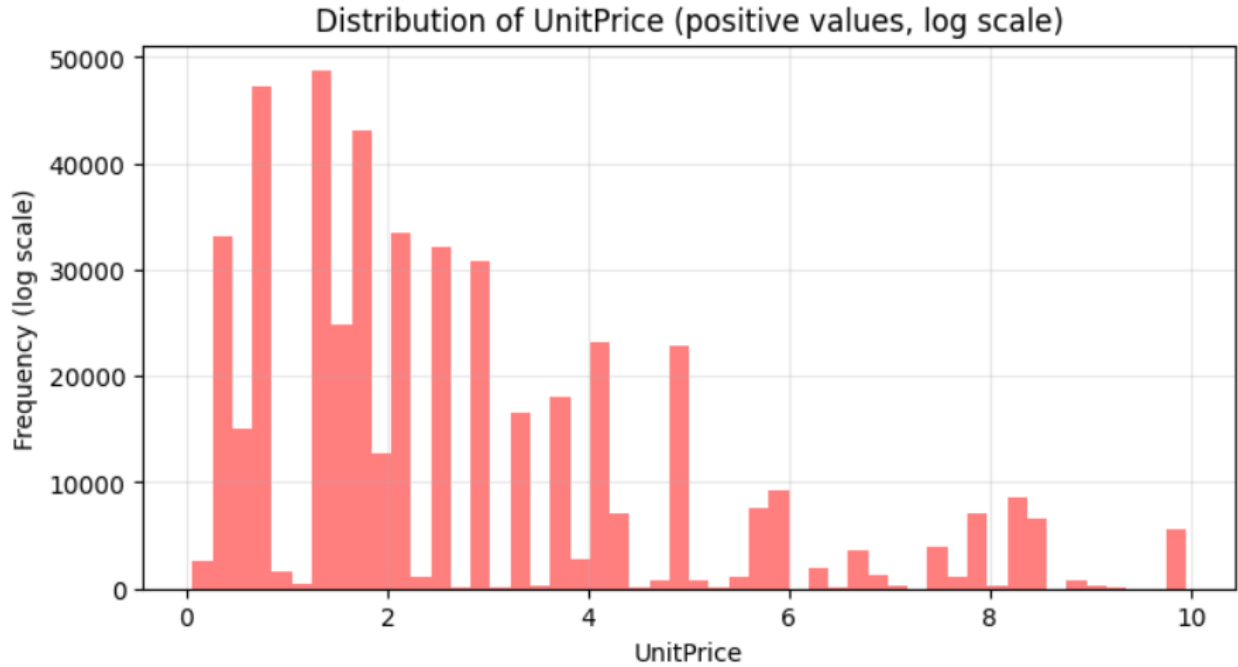


Mean products per transaction: 25.252207666181466
Median products per transaction: 15.0
Max products per transaction: 1108

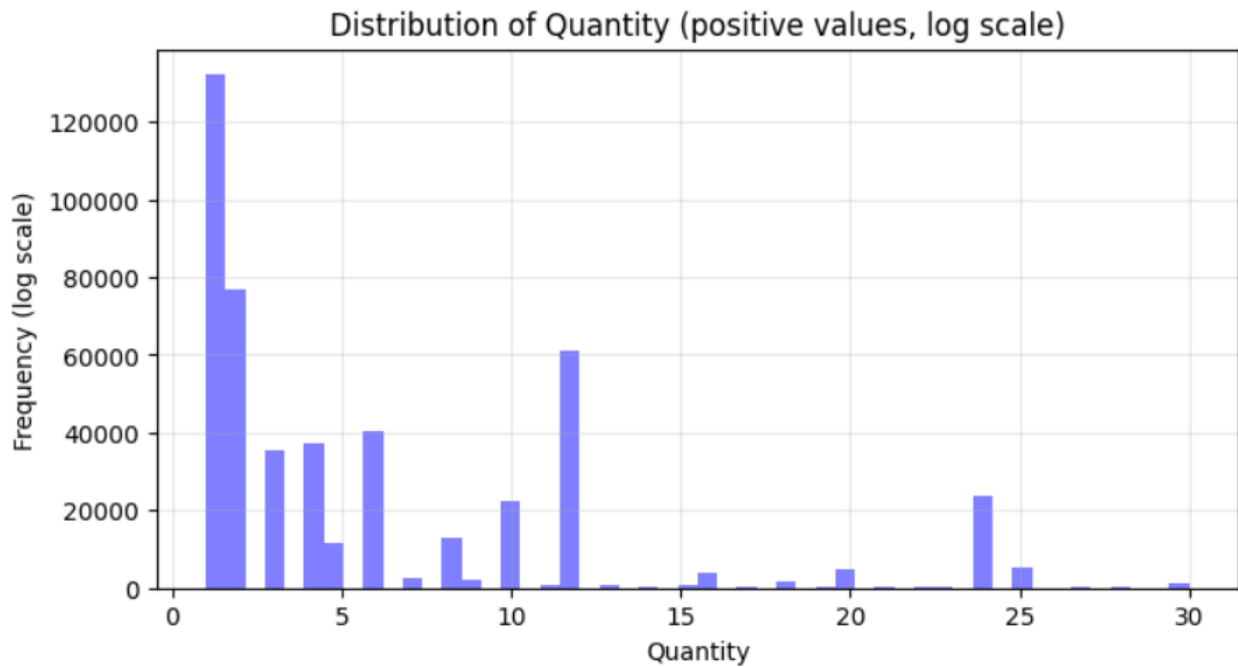
Plotting the top 15 most frequent items:



Distribution of UnitPrice:



The distribution is rather skewed which makes sense because items with lower prices have been bought more than the expensive ones.



The distribution of quantity is rather skewed.

Association Rule Mining

First, we find all possible rules through Apriori algorithm:

```
transactions = baskets["Description"].apply(
    lambda basket: [str(item) for item in basket]
).tolist()
te = TransactionEncoder()

te_array = te.fit(transactions).transform(transactions)
df_hot = pd.DataFrame(te_array, columns=te.columns_)
df_hot
```

	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
20605	False	False	False	False	False	False	False	False	False
20606	False	False	False	False	False	False	False	False	False
20607	False	False	False	False	False	False	False	False	False
20608	False	False	False	False	False	False	False	False	False
20609	False	False	False	False	False	False	False	False	False

20610 rows × 4207 columns

I tried various values for support, confidence, and lift thresholds:

Support:

The value of support will be later chosen from [0.01, 0.03] as 0.03 is the largest support that provides more than 10 rules.

0.01 3200

0.02 180

0.03 18

Confidence:

0.5 → 841

0.6 → 503

0.7 → 265

0.8 → 105

0.9 → 20

The confidence threshold will be later chosen from [0.5, 0.9).

Lift:

I picked the lift threshold as 1 since lift > 1 means that the items are correlated with each other.

The lift values are quite high and since the data matrix is sparse, it rings a bell that there is a possibility that the dataset is full of null transactions and since lift is sensitive to such transactions, it can mislead us. Therefore, I will consider some other measure to verify the rules' interestingness later.

I added more measures to the rules data frame below, such as kulc and imbalance ratio and try to filter out some using these measures' thresholds.

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski	Imbalance_Ratio
83	(herb marker parsley, herb marker thyme)	(herb marker rosemary)	0.010792	0.012161	0.010286	0.953052	78.372613	1.0	0.010155	21.040981	0.998011	0.812000	0.952474	0.899442	0.1080
100	(herb marker thyme, herb marker basil)	(herb marker rosemary)	0.010640	0.012161	0.010134	0.952381	78.317460	1.0	0.010004	20.744629	0.997849	0.800000	0.951795	0.892857	0.1200
25	(regency tea plate roses, regency tea plate pink)	(regency tea plate green)	0.013630	0.019305	0.012921	0.947955	49.104587	1.0	0.012657	18.843357	0.993172	0.645570	0.946931	0.808623	0.2835
84	(herb marker parsley, herb marker rosemary)	(herb marker thyme)	0.010894	0.012009	0.010286	0.944186	78.626396	1.0	0.010155	17.701514	0.998155	0.815261	0.943508	0.900363	0.0884
101	(herb marker rosemary, herb marker basil)	(herb marker thyme)	0.010843	0.012009	0.010134	0.934579	77.826413	1.0	0.010004	15.102156	0.997972	0.796813	0.933784	0.889231	0.0916

Filtering rules:

```
filtered_rules = rules[
    (rules["lift"] > 1) &
    (rules["support"] > 0.01) &
    (rules["confidence"] >= 0.5) &
    (rules["Imbalance_Ratio"] <= 0.5) &
    (rules["kulczynski"] >= 0.5)
]

filtered_rules = filtered_rules.sort_values("lift", ascending=False)
print(len(filtered_rules))
top10_rules = filtered_rules.head(10)
top10_rules
```

✓ 0.0s

I fine tuned the threshold values to ensure all of them are considered as well as the fact that the number of resulting rules reaches the minimum of 10.

I chose the kulc threshold as 0.5 since less values of kulc mean that the rule is not reliable enough and the 2 items are not much correlated.

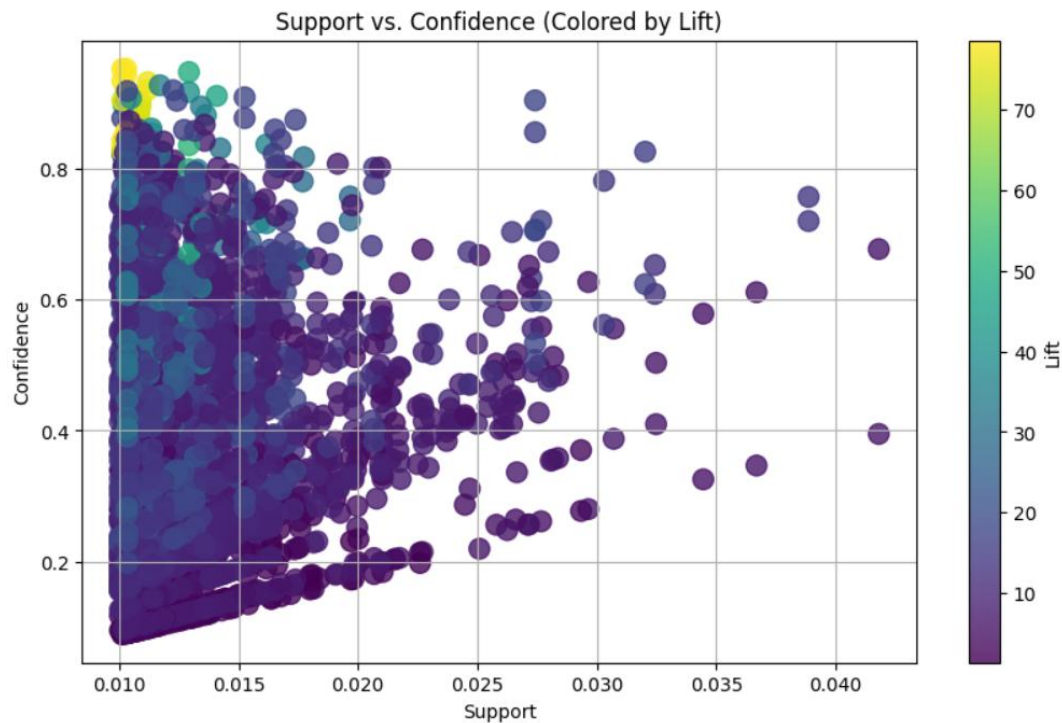
Moreover, I chose the threshold of 0.59 for imbalance ratio to ensure that the correlation between rules' items is not one-sided.

Final 10 rules:

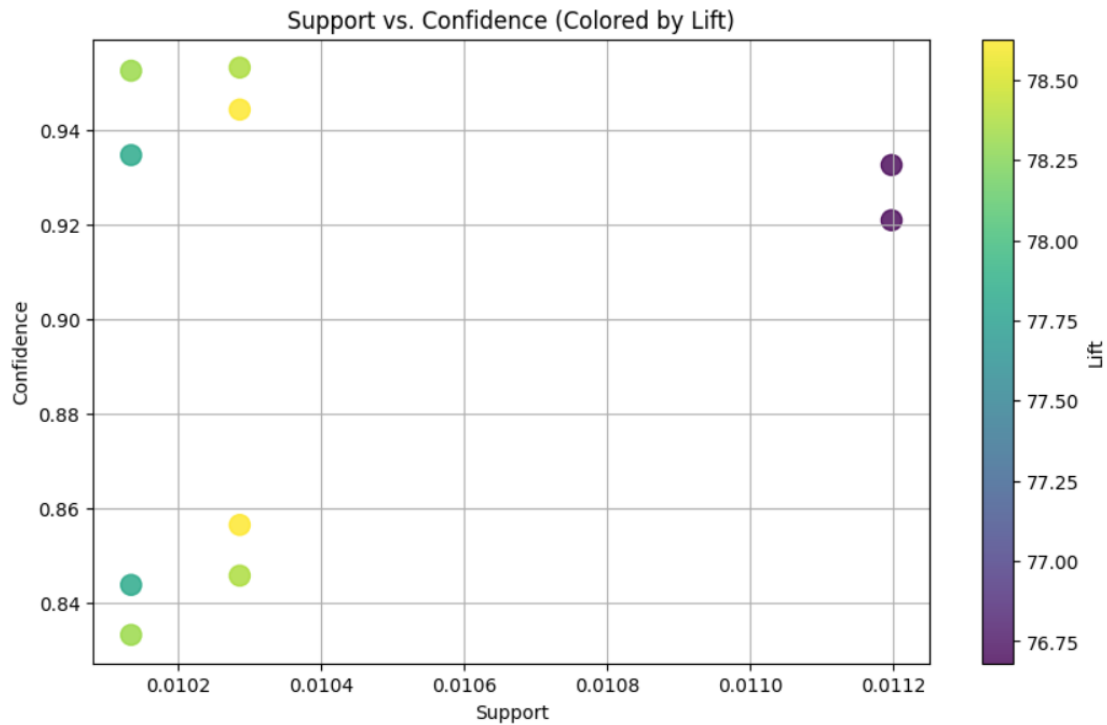
	antecedents	consequents	support	confidence	lift
87	(herb marker thyme)	(herb marker parsley, herb marker rosemary)	0.010286	0.856540	78.626396
84	(herb marker parsley, herb marker rosemary)	(herb marker thyme)	0.010286	0.944186	78.626396
83	(herb marker parsley, herb marker thyme)	(herb marker rosemary)	0.010286	0.953052	78.372613
88	(herb marker rosemary)	(herb marker parsley, herb marker thyme)	0.010286	0.845833	78.372613
100	(herb marker thyme, herb marker basil)	(herb marker rosemary)	0.010134	0.952381	78.317460
103	(herb marker rosemary)	(herb marker thyme, herb marker basil)	0.010134	0.833333	78.317460
102	(herb marker thyme)	(herb marker rosemary, herb marker basil)	0.010134	0.843882	77.826413
101	(herb marker rosemary, herb marker basil)	(herb marker thyme)	0.010134	0.934579	77.826413
45	(herb marker thyme)	(herb marker rosemary)	0.011198	0.932489	76.681716
46	(herb marker rosemary)	(herb marker thyme)	0.011198	0.920833	76.681716

The shape of the scatter plot of rules before and after filtering:

Before:



After:



As can be seen clearly, the axis of the plot has changed and the higher values of each of the measures have survived which is quite desired and a satisfactory.

Top 5 Rules

I designed a formula and weighted each of the required measures (support, confidence, and lift) to ensure all of them contribute to the very last selection:

```
top10_rules["biz_score"] = (  
    top10_rules["lift"] * 0.5 +  
    top10_rules["confidence"] * 0.3 +  
    top10_rules["support"] * 0.2  
)  
  
best5 = top10_rules.sort_values("biz_score", ascending=False).head(5)
```

Final 5 rules:

```

                                antecedents \
84  (herb marker parsley, herb marker rosemary)
87                                (herb marker thyme)
83  (herb marker parsley, herb marker thyme)
100 (herb marker thyme, herb marker basil)
88                                (herb marker rosemary)

                                consequents  support  confidence \
84                                (herb marker thyme) 0.010286  0.944186
87  (herb marker parsley, herb marker rosemary) 0.010286  0.856540
83                                (herb marker rosemary) 0.010286  0.953052
100 (herb marker rosemary) 0.010134  0.952381
88  (herb marker parsley, herb marker thyme) 0.010286  0.845833

                                lift
84  78.626396
87  78.626396
83  78.372613
100 78.317460
88  78.372613
```

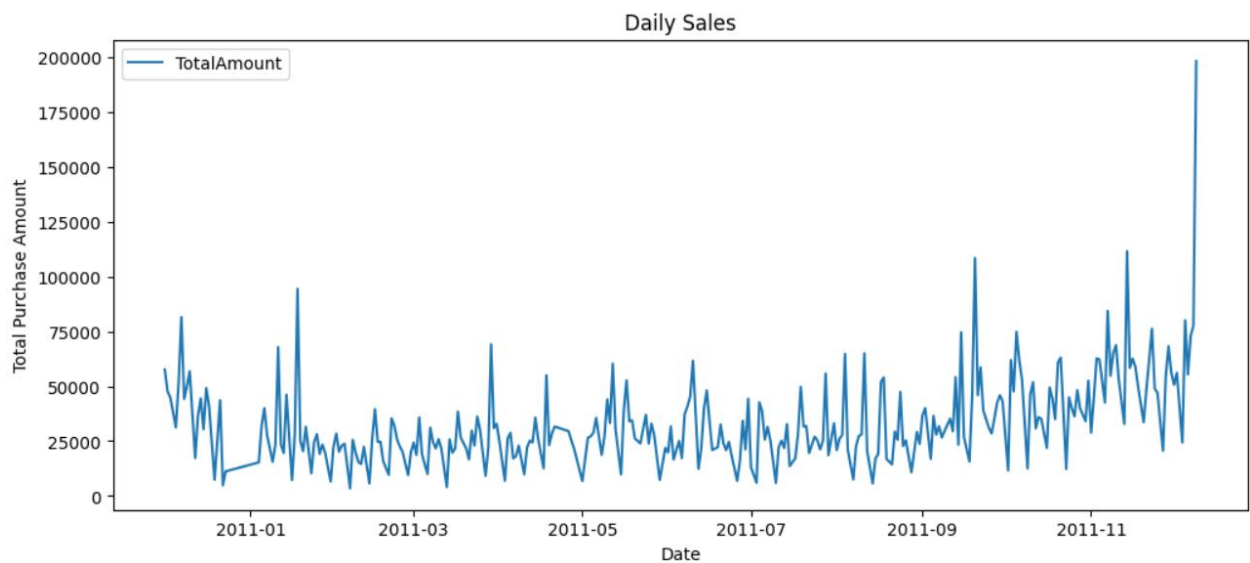
Time Series Analysis

First, I grouped the daily sales:

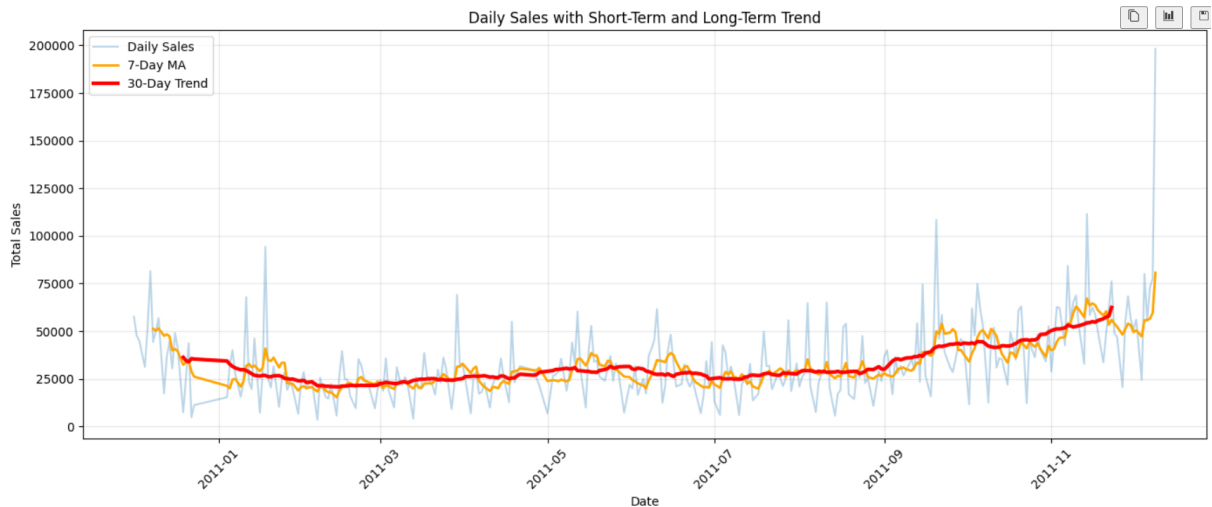
```
df["InvoiceDay"] = df["InvoiceDate"].dt.date
daily_sales = df.groupby("InvoiceDay")["TotalAmount"].sum().reset_index()
daily_sales.columns = ["Date", "TotalAmount"]
daily_sales = daily_sales.sort_values("Date")

daily_sales.plot(x="Date", y="TotalAmount", kind="line", figsize=(12,5))
plt.ylabel("Total Purchase Amount")
plt.title("Daily Sales")
plt.show()
```

And then plotted the daily sales in a chronological order:



As can be seen there are a lot of fluctuations in the daily sales, however, they are going to be smoothed out using simple moving average method in the next step.



I plotted 2 moving average lines with lengths of 7 and 30 and as expected the more the SMA parameter increases, the smoother the line becomes.

Time Series Forecasting

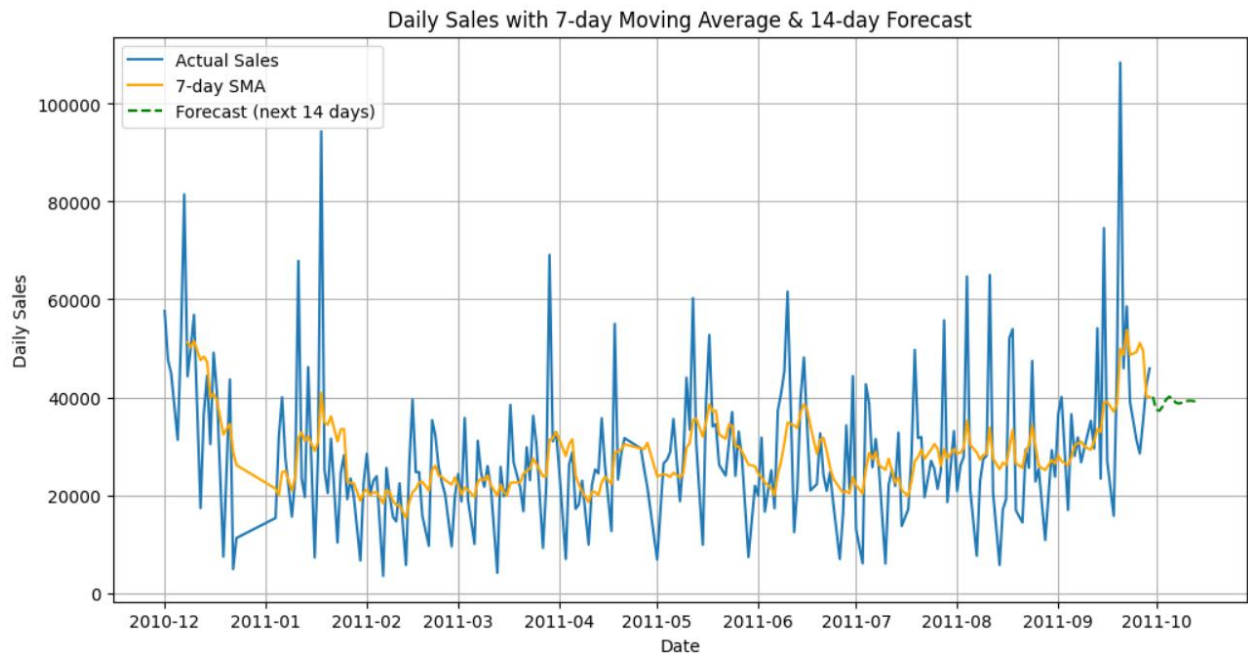
Splitting train and test data:

```
daily_sales["Date"] = pd.to_datetime(daily_sales["Date"])
ts = daily_sales.set_index("Date")["TotalAmount"]

split_idx = int(len(ts) * 0.8)
train = ts.iloc[:split_idx]
test = ts.iloc[split_idx:]

window_size = 7
forecast_days = 14
```

I applied SMA as the forecasting method; the green dashed line shows the prediction:



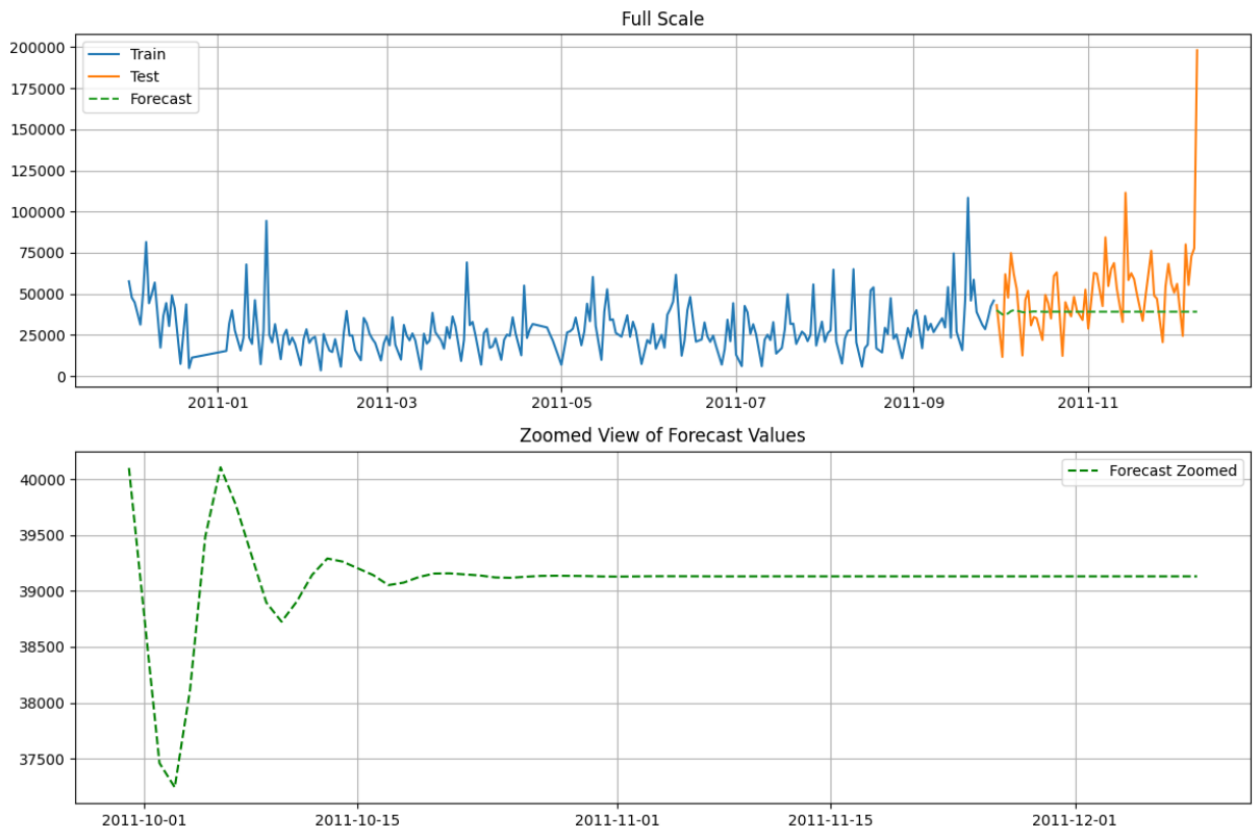
Verifying the method using the test data

Important note:

There are 2 cases of prediction generations using SMA method; at each timestamp there are 2 possibilities: adding the predicted value to the window or replacing the actual value.

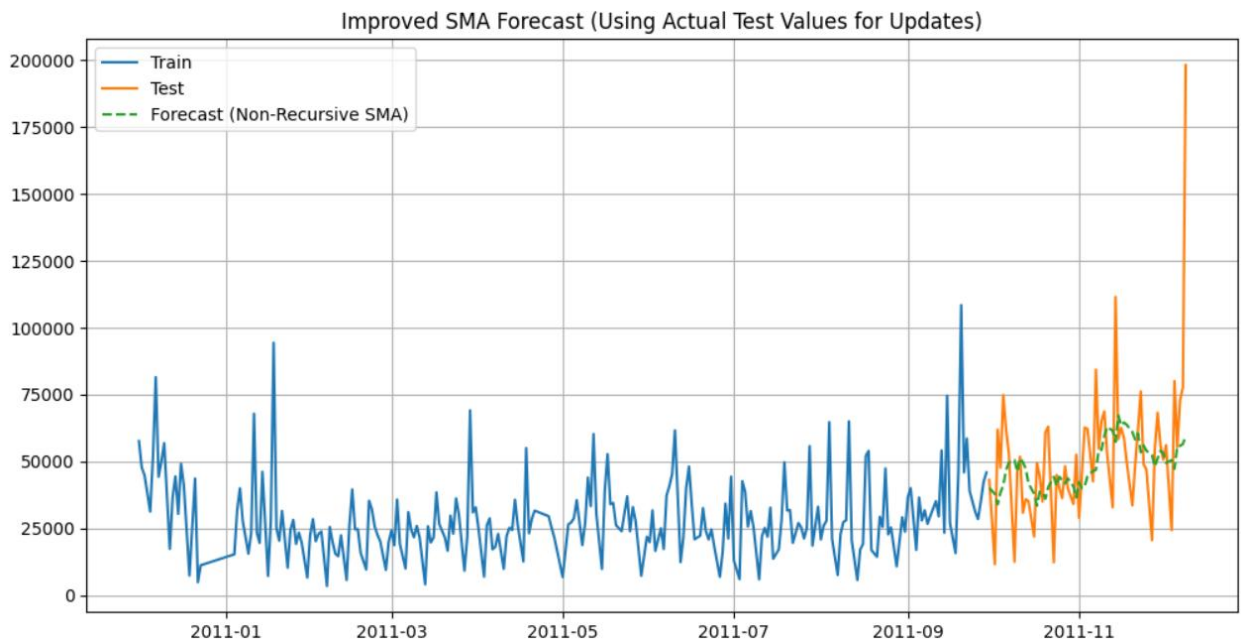
The point is that MA is a smoothing method and after 2 iterations of window size it completely flattens. However, replacing the actual values avoids that kind of flattening and improves accuracy a little bit. Below comes both cases and as can be seen in the case of no replacement, the method made more realistic predictions.

Testing without adding the actual data:



MAE : 19607.19062258237
RMSE: 29521.287093992727
MAPE: 40.1250740327558

Testing with the actual data:




```
MAE : 16107.386674473066  
RMSE: 25402.904763274044  
MAPE: 39.73200669653199
```

The error measures of the 2 approaches are quite expected as after adding the actual data points the error measures have decreased.

Analysis and Interpretation

SMA is one of the least reliable forecasting methods for anything beyond very short-term prediction and the flattening we observed is exactly why.

Reason 1: SMA cannot model trend

If data is trending upward or downward:

SMA smooths it out, forecast stays “in the middle” and completely misses the trend

Reason 2: SMA cannot model seasonality

If sales have weekly patterns (Mon–Sun cycle), SMA ignores this.

Reason 3: Recursive SMA always collapses to a constant

Recursive forecasting means:

Predict tomorrow using today's forecast, Predict next day using today's predicted value

Evaluation Metrics:

$$\text{MAE} = (1/n) \sum_{i=1}^n |y_i - \hat{y}_i|$$

mean squared error (MSE), the average squared difference between the value observed in a statistical study and the values predicted from a model.

It provides an estimate of the size of the inaccuracy, but not its direction (e.g., over or under-prediction).

Resiliency to outliers. The MAE is not as impacted by extreme results as other metrics, such as Mean Squared Error (MSE), are. This makes it an appropriate measure for datasets that include outliers or extreme values.

RMSE = $\sqrt{(\sum(P_i - O_i)^2) / n}$

It is one of the standard ways to measure the error rate of a model in predicting quantitative data.

It is a metric predominantly utilized in regression analysis and forecasting, where accuracy matters significantly. The lower the RMSE, the better the model's ability to predict accurately. Conversely, a higher RMSE signifies a greater discrepancy between the predicted and actual outcomes.

MAPE

Mean absolute percentage error measures the average magnitude of error produced by a model, or how far off predictions are on average. A MAPE value of 20% means that the average absolute percentage difference between the predictions and the actuals is 20%

$$\text{MAPE} = n \sum (|\text{Actual}_i - \text{Forecast}_i| / \text{Actual}_i) \times 100$$